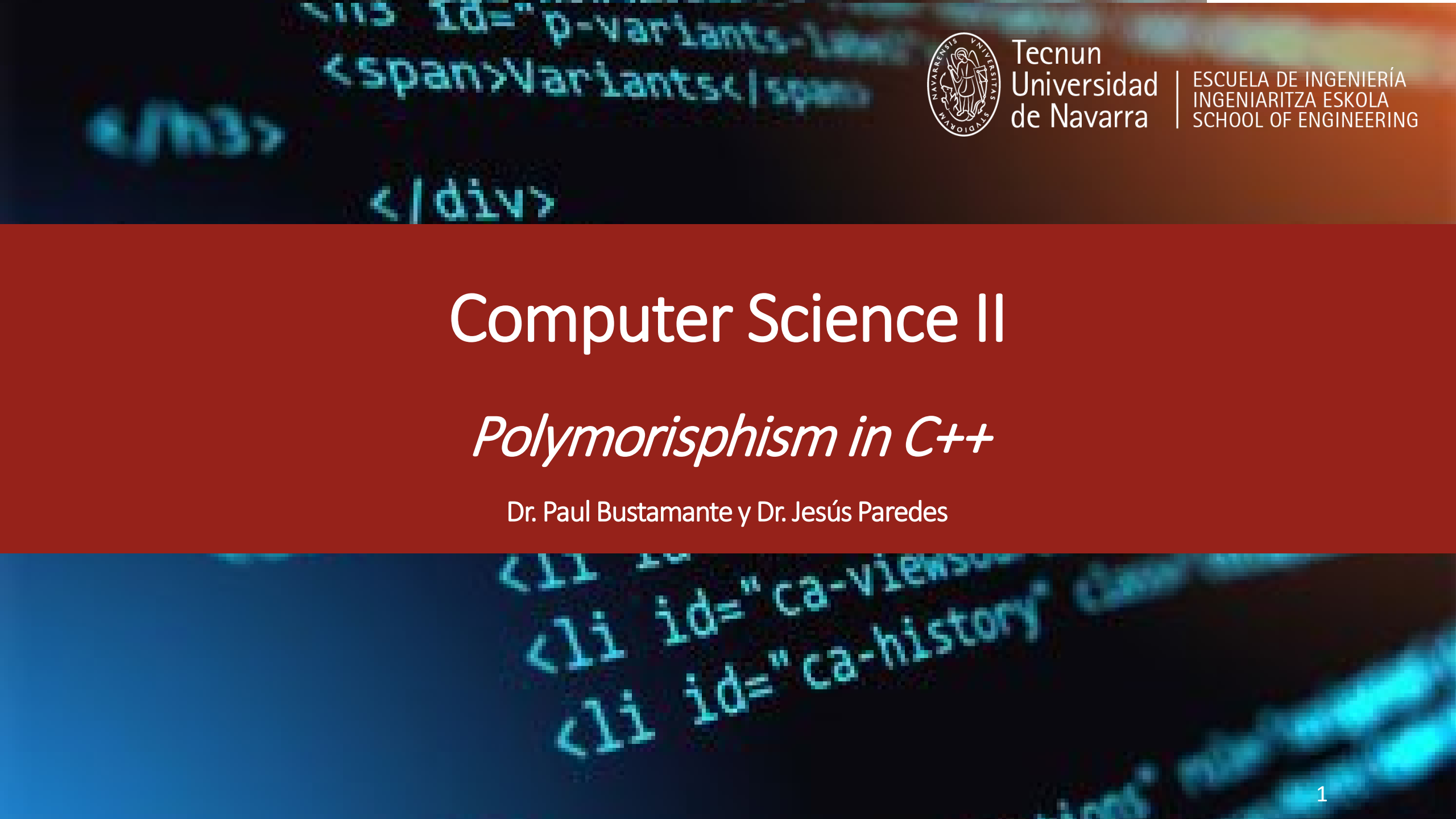


Tecnun
Universidad
de Navarra

ESCUELA DE INGENIERÍA
INGENIARITZA ESKOLA
SCHOOL OF ENGINEERING

Computer Science II

Polymorisphism in C++

Dr. Paul Bustamante y Dr. Jesús Paredes

Index

Review: Inheritance

Why

- Classes→ Group information
- Inheritance → Write less code → inherits variables and functions

How to set inheritance

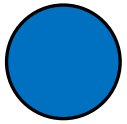
```
class Derived : public Base
```

Who can access to functions/variables:

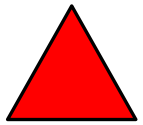
	public	protected	private
Main			
Derived class			

Review: Without Inheritance

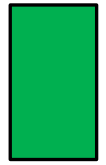

```
// Circle.h
class Circle{
  double area, perimeter, radius;
public:
  Circle(double a=0, double p=0, double r=0);
  ~Circle();
  void prt();
  void Calculate(void);
};
```



```
// Triangle.h
class Triangle{
  double area, perimeter, base, height;
public:
  Triangle(double a=0, double p=0, double b=0,
double h=0);
  ~Triangle();
  void prt();
};
```



```
// Rectangle.h
class Rectangle{
  double area, perimeter, base, height;
public:
  Rectangle(double a=0, double p=0, double b=0,
double h=0);
  ~Rectangle();
  void prt();
  void Calculate (void);
};
```



```
// Circle.cpp
Circle::Circle(double a, double p, double r) {
  area = a;
  perimeter = p;
  radius = r;
}

Circle::~Circle() {}

void Circle::prt() {
  cout << "Area: " << area;
  cout << "\tPerimeter: " << perimeter;
  cout << "\tRadius: " << radius << endl;
}

void Circle::Calculate (void) {
  area = 3.14 * radius * radius;
  perimeter = 2 * 3.14 * radius;
}
```

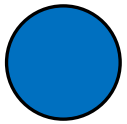
```
// Triangle.cpp
Triangle::Triangle(double a, double p, double
b, double h) {
  area = a;
  perimeter = p;
  base = b;
  height = h;
}
Triangle::~Triangle() {}

void Triangle::prt() {
  cout << "Area: " << area;
  cout << "\tPerimeter: " << perimeter;
  cout << "\tBase: " << base;
  cout << "\tHeight: " << height << endl;
}
```

```
// Rectangle.cpp
Rectangle::Rectangle(double a, double p, double
b, double h) {
  area = a;
  perimeter = p;
  base = b;
  height = h;
}
Rectangle::~Rectangle() {}

void Rectangle::prt() {
  cout << "Area: " << area;
  cout << "\tPerimeter: " << perimeter;
  cout << "\tBase: " << base;
  cout << "\tHeight: " << height << endl;
}

void Rectangle::Calculate (void) {
  area = base * height;
  perimeter = 2 * (base + height); }
```

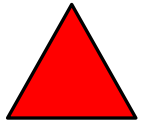
Review: Without Inheritance

// The same, but all in ".h" file

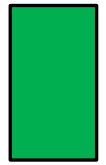
```
class Circle{
  double area, perimeter, radius;
public:
  Circle(double a=0, double p=0, double r=0) {
    area = a;
    perimeter = p;
    radius = r;
  }

  ~Circle() {}

  void prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tRadius: " << radius << endl;
  }

  void Calculate(void) {
    area = 3.14 * radius * radius;
    perimeter = 2 * 3.14 * radius;
  }
};
```



```
class Triangle{
  double area, perimeter, base, height;
public:
  Triangle(double a=0, double p=0, double b=0,
  double h=0) {
    area = a;
    perimeter = p;
    base = b;
    height = h;
  }

  ~Triangle() {}

  void prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tBase: " << base;
    cout << "\tHeight: " << height << endl;
  }
};
```



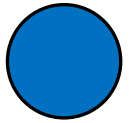
```
class Rectangle{
  double area, perimeter, base, height;
public:
  Rectangle(double a=0, double p=0, double b=0,
  double h=0) {
    area = a;
    perimeter = p;
    base = b;
    height = h;
  }

  ~Rectangle() {}

  void prt() {
    cout << "Area: " << area;
    cout << "\tPerimeter: " << perimeter;
    cout << "\tBase: " << base;
    cout << "\tHeight: " << height << endl;
  }

  void Calculate (void) {
    area = base * height;
    perimeter = 2 * (base + height);
  }

};
```



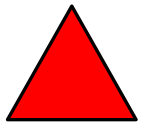
Review: Using Inheritance (1)


```
class Shape{
protected:
   double area;
   double perimeter;
public:
   Shape(double a=0, double p=0) { area = a; perimeter = p; }
   ~Shape(){}
   void prt() {
      cout << "Area: " << area;
      cout << "\tPerimeter: " << perimeter;
   }
   void Calculate(){ cout << "Not defined" << endl;}
};
```

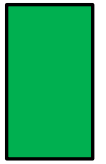
```
class Circle : public Shape{
   double radius;
public:
   Circle(double a=0, double p=0, double r=0) :Shape(a,p) { radius = r; }
   ~Circle() {}
   void prt() {
      Shape::prt();
      cout << "\tRadius: " << radius << endl;
   }
   void Calculate(void) {
      area = 3.14 * radius * radius;
      perimeter= 2 * 3.14 * radius;
   }
};
```



```
class Triangle : public Shape {
   double base, height;
public:
   Triangle(double a=0, double p=0, double b=0, double h=0) {
      base = b;
      height = h;
   }
   ~Triangle(){}
   void prt() {
      Shape::prt();
      cout << "\tBase: " << base << "\tHeight: " << height << endl;
   }
};
```



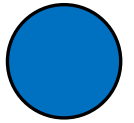
```
class Rectangle : public Shape{
   double base, height;
public:
   Rectangle(double a=0, double p=0, double b=0, double h=0):Shape(a, p){
      base = b;
      height = h;
   }
   ~Rectangle() {}
   void prt() {
      Shape::prt();
      cout << "\tBase: " << base << "\tHeight: " << height << endl;
   }
   void Calculate (void) {
      area = base * height;
      perimeter = 2 * (base + height);
   }
};
```



Review: Using Inheritance (2)


```
class Shape{
protected:
   double area;
   double perimeter;
public:
   Shape(double a=0, double p=0) { area = a; perimeter = p; }
   ~Shape(){}
   void prt() {
      cout << "Area: " << area;
      cout << "\tPerimeter: " << perimeter;
   }
   void Calculate(){ cout << "Not defined" << endl;}
};
```

```
class Circle : public Shape{
   double radius;
public:
   Circle(double a=0, double p=0, double r=0) :Shape(a,p) { radius = r; }
   ~Circle() {}
   void prt() {
      Shape::prt();
      cout << "\tRadius: " << radius << endl;
   }
   void Calculate(void) {
      area = 3.14 * radius * radius;
      perimeter= 2 * 3.14 * radius;
   }
};
```

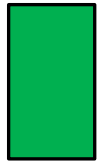


```
class Triangle : public Shape {
   double base, height;
public:
   Triangle(double a=0, double p=0
      base = b;
      height = h;
   }
   ~Triangle(){}
   void prt() {
      Shape::prt();
      cout << "\tBase: " << base <<
   }
};
```

Not defined in Triangle

```
#include "Shape.h"
void main()
{
    Shape s;
    Circle c;
    Rectangle r;
    Triangle t;

    s.Calculate();
    c.Calculate();
    r.Calculate();
    t.Calculate();
}
```

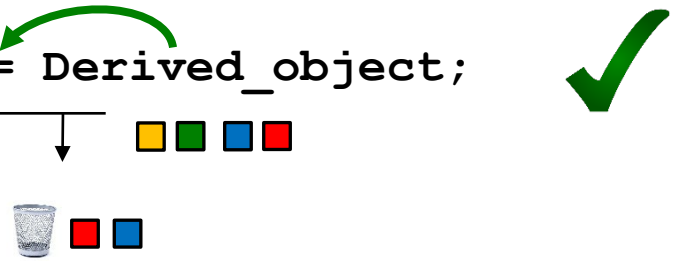
```
class Rectangle : public Shape{
   double base, height;
public:
   Rectangle(double a=0, double p=0, double b=0, double h=0):Shape(a, p){
      base = b;
      height = h;
   }
   ~Rectangle() {}
   void prt() {
      Shape::prt();
      cout << "\tBase: " << base << "\tHeight: " << height << endl;
   }
   void Calculate (void) {
      area = base * height;
      perimeter = 2 * (base + height);
   }
};
```



Conversions between objects of base classes and derived classes

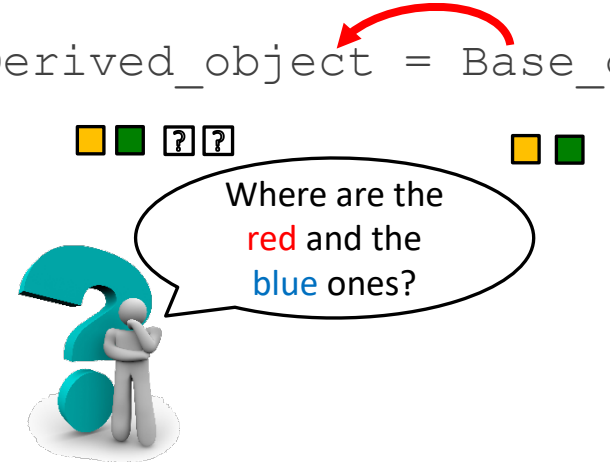
It is possible to make ***conversions*** or ***assignments*** between an object of a ***derived class*** to an object of the ***base class*** (it is possible to go from the most particular to the most general, although information may be lost). It is possible to do:

```
Base_object = Derived_object;
```



Conversions in the opposite direction (from the most general to the most particular) ***are not possible***, as values are not available for all member variables of the derived class and there would be variables that would remain uninitialized).

```
Derived_object = Base_object;
```




```
#include "Shape.h"
void main(){
    Shape s1, s2, s3;
    Circle c;
    Rectangle r;
    Triangle t;
    s1 = c; s1.prt();
    s2 = r; s2.prt();
    s3 = t; s3.prt();
    // The console output?
}
```

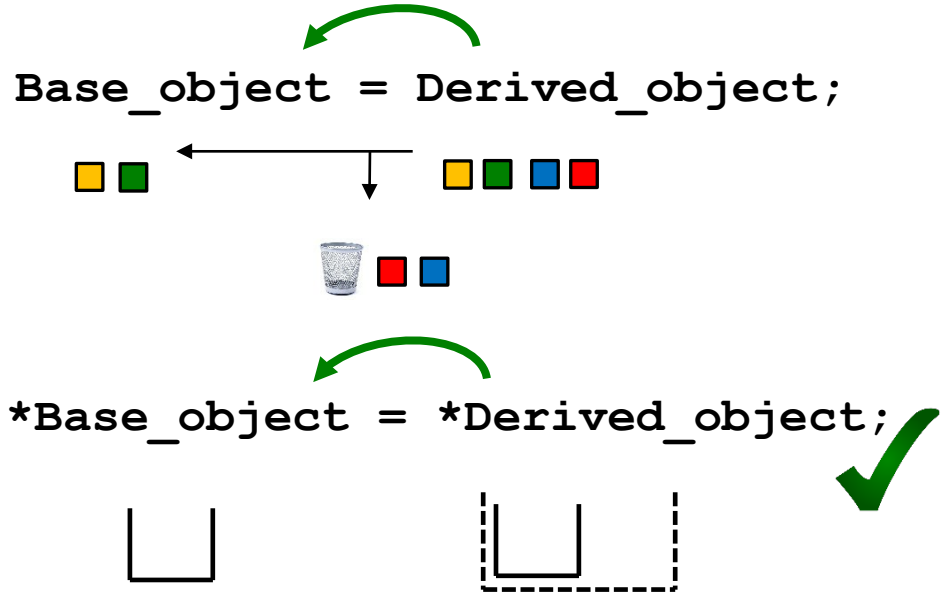
Conversions between objects of base classes and derived classes

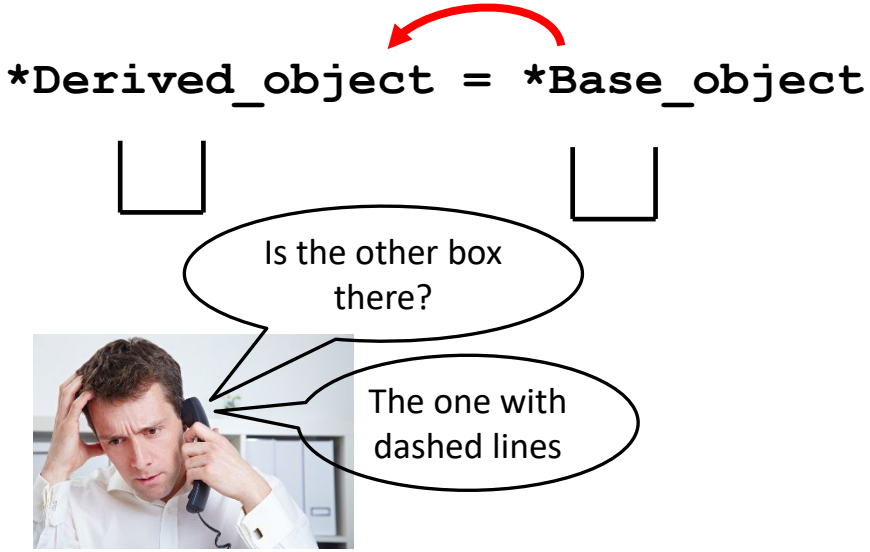
Similarly, a ***pointer to a derived class*** can be converted into a ***pointer to the base class***.

A pointer to the base class can store the address of an object of any of the classes derived from that base class.

An object of a derived class can be referenced with a pointer to the base class.

When an object is referred to by means of a pointer, the ***type of pointer*** determines the ***member function*** that applies to that object, if such a function is defined in the ***base*** and ***derived classes***.





Review: Conversions between objects of base classes and derived classes


```
Shape my_shape_1(2, 2);
Rectangle my_rectangle_1(12,14,3,4);
Shape my_shape_2;

my_shape_2 = my_rectangle_1;
my_rectangle_1.prt();
my_shape_2.prt();
```

```
//NOT ALLOWED
Rectangle my_rectangle_2;
my_rectangle_2 = my_shape_2;
```

NAME	VALUE	TYPE	SIZE
my_shape	**{area=2 perimeter=2}**	Shape	**16**
area	2	double	8
perimeter	2	double	8
my_rectangle_1	**{area=-9.2559631349317831e+61 perimeter=-9.2559631349317831e+61 base=3 height=4}**	Rectangle	**48**
Shape	{area=12 perimeter=14}	Shape	16
area	-9.2559631349317831e+61	double	8
perimeter	-9.2559631349317831e+61	double	8
base	3	double	8
height	4	double	8
my_shape_2	**{area=12.000000000000000 perimeter=14.000000000000000 }**	Shape	**16**
area	12	double	8
perimeter	14	double	8

Review: Pointers to *base class* to use objects of *derived classes*

In a ***class hierarchy*** with functions of the ***same name*** defined in all classes, the ***object type*** determines which function is applied.

- For example: with ***objects name***:

```
salesman1.calc_payment();
worker3.calc_payment();
```

with ***pointer to objects***:

```
manager *pntr1;
 pntr1->calc_payment();
```

A function that expects a ***pointer to the base class*** can be passed a ***pointer to any of its derived classes*** (by the laws of pointer conversion).

In this way, a ***list of objects of different classes*** can be treated with the interface - with the member functions - of the ***base class***.

This treatment is ***generic***, without taking into account the peculiarities of ***derived classes***, because if a ***pointer to a base class*** is used with an object of a ***derived class***, the function of the ***base class*** is used.

Customised treatment with ***generic*** pointers (to objects of the base class) is achieved by means of ***polymorphism***.

Review: Pointers to *base class* to use objects of *derived classes*


```
Shape single_shape(2, 2);
Shape static_shape_list[10];
Shape* shape_pointer1 = &single_shape;
Shape* shape_pointer2 = new Shape(3,3);
static_shape_list[0] = single_shape;
```

```
static_shape_list[0].prt();
shape_pointer1->prt();
shape_pointer2->prt();
(&single_shape)->prt();
```

NAME	VALUE	TYPE	SIZE
single_shape	{area=2 perimeter=2}	Shape	16
&single_shape	**0x0098f684** {area=2 perimeter=2}	Shape*	4
*single_shape	no operator "*" matches these operands		

static_shape_list	0x0098f5dc {{{area=2 perimeter=2}, {area=0 perimeter=0}, ...}	Shape[10]	160
&static_shape_list	0x0098f5dc {{area=2 perimeter=2 }, {area=0 perimeter=0}, ...}	Shape[10]*	4
*static_shape_list	{area=2 perimeter=2}	Shape	16

shape_pointer1	**0x0098f684** {area=2 perimeter=2}	Shape*	4
&shape_pointer1	0x0098f5d0 {0x0098f684 {area=2 perimeter=2}}	Shape**	4
*shape_pointer1	{area=2 perimeter=2}	Shape	16

shape_pointer2	0x00cf6710 {area=3 perimeter=3}	Shape*	4
&shape_pointer2	??? {0x00cf6710 {area=3 perimeter=3 }}	Shape**	4
*shape_pointer2	{area=3 perimeter=3}	Shape	16

Polymorphism

POLYS (πολύς=many) + MORFOS (μορφή=form)

Polymorphism in OOP is the ability to call different functions with the same name:

The word ***polymorphism*** refers to the ability to call member ***functions***:

- with a ***single name*** and an analogous but ***different mission***
- acting on ***different objects*** in a class hierarchy
- that are named in the precise way ***without having to specify*** the exact type of the objects

Applied to functions in two ways

- By ***function overloading***
  - Same type of output variable
  - Same function name
  - Different number/types of input variables
- By ***Inheritance***
  - Same type of output variable
  - Same function name
  - Same number/types of input variables

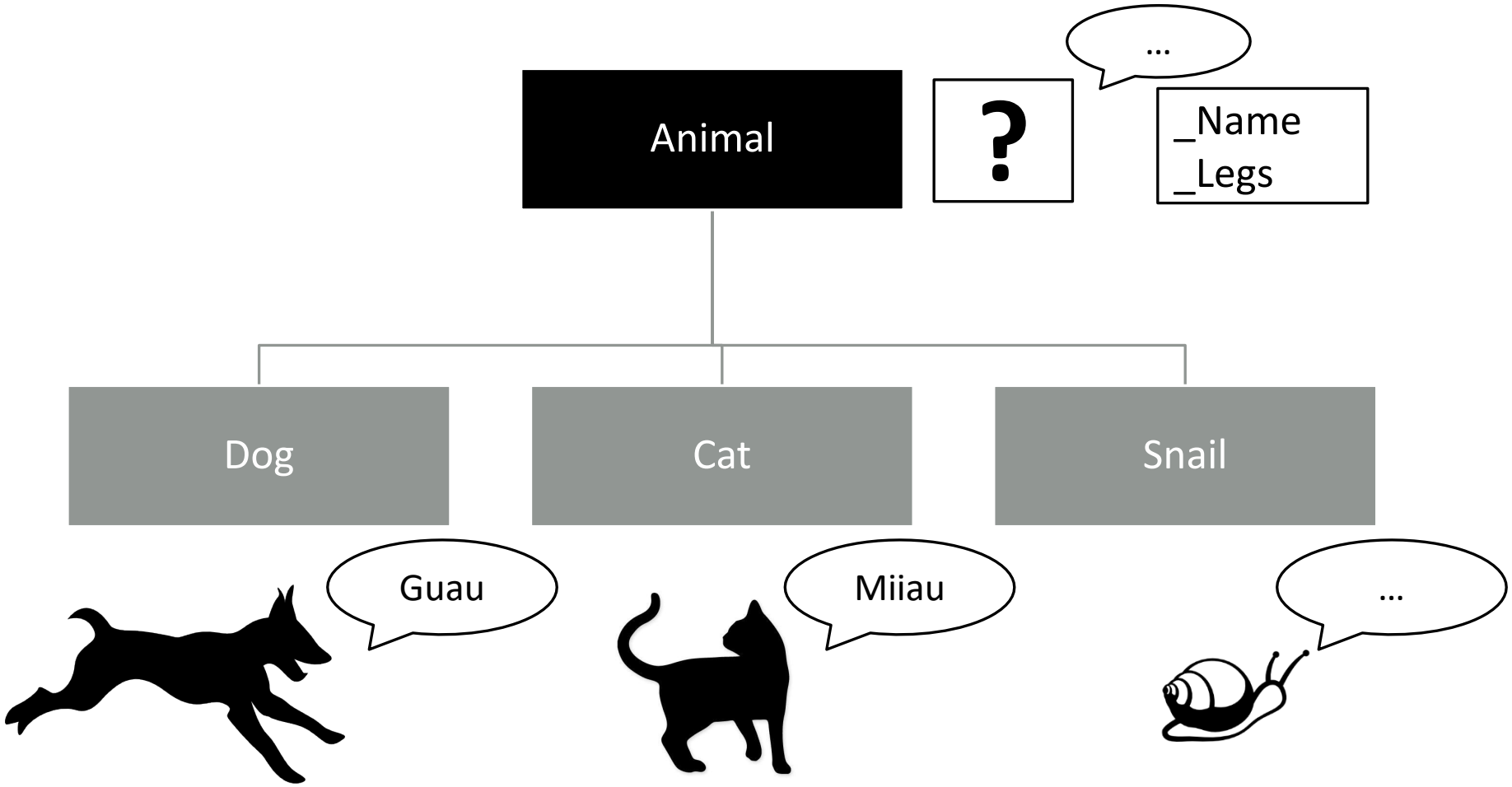
Polymorphism by function overloading


```
// Different type of input variables
double product(int a, double b) { return a * b; }
double product(double a, int b) { return a * b; }
double product(double a, double b) { return a * b; }

//  Different number of input variables
double product(double a, double b) { return a * b; }
double product(double a, double b, double c) { return a * b * c; }

// Different type of output variable → NOT ALLOWED
int product(double a, double b) { return a * b; }
double product(double a, double b) { return a * b; }
```

Allowed **if** each function belongs to a different workspace

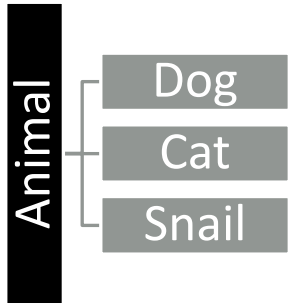
```
namespace space1 {
    int product(double a, double b) {
        return (int)(a * b);
    }
}
namespace space2 {
    double product(double a, double b)  {
        return (a * b);
    }
}
```

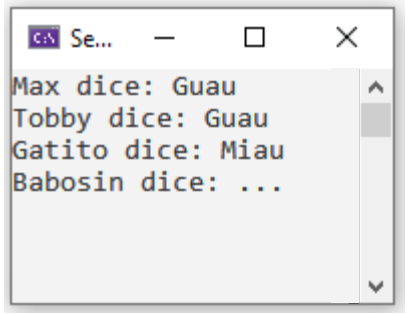
Polymorphism by inheritance



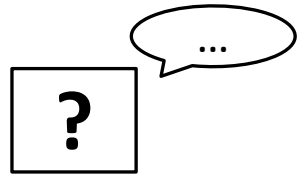
Inheritance without polymorfism


```
void main()
{
  Dog animal_1(4, (char*)"Max");
  Dog animal_2(3, (char*)"Tobby");
  Cat animal_3(4, (char*)"Gatito");
  Snail animal_4(0, (char*)"Babosin");

  cout << animal_1.GetName() << " dice: ";
  animal_1.Speak();
  cout << animal_2.GetName() << " dice: ";
  animal_2.Speak();
  cout << animal_3.GetName() << " dice: ";
  animal_3.Speak();
  cout << animal_4.GetName() << " dice: ";
  animal_4.Speak();
}
```





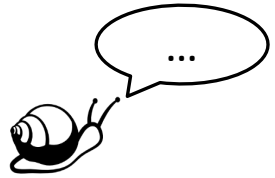

```
class Animal{
protected:
  char *_name;
  int _legs;
public:
  Animal(int legs = 0, char* name=(char*)""){ _legs = legs; _name = name; }
  char* GetName() {return _name;}
  void Speak() {cout << "..." << endl; }
};
```




```
class Dog : public Animal{
public:
  Dog(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Guau" << endl; }
};
```

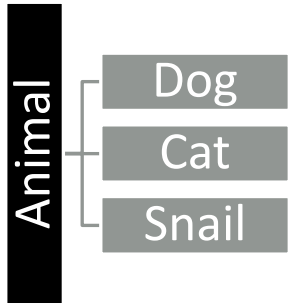

```
class Cat : public Animal{
public:
  Cat(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Miau" << endl; }
};
```

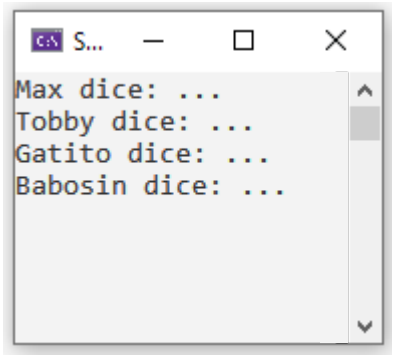

```
class Snail : public Animal{
public:
  Snail(int legs, char* name) :Animal(legs, name){}
};
```



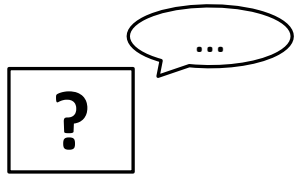
Inheritance without polymorfism


```
void main()
{
  Dog animal_1(4, (char*)"Max");
  Dog animal_2(3, (char*)"Tobby");
  Cat animal_3(4, (char*)"Gatito");
  Snail animal_4(0, (char*)"Babosin");

  //Animal animal_list[10] // OPTION 1
  Animal *animal_list = new Animal[10]; // OPTION 2
  animal_list[0] = animal_1;
  animal_list[1] = animal_2;
  animal_list[2] = animal_3;
  animal_list[3] = animal_4;
  for (int i = 0; i < 4; i++)
  {
     cout << animal_list[i].GetName() << " dice: " ;
     animal_list[i].Speak();
  }
}
```





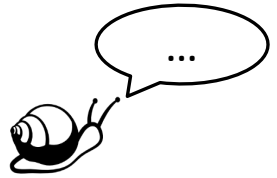

```
class Animal{
protected:
  char *_name;
  int _legs;
public:
  Animal(int legs = 0, char* name=(char*)""){ _legs = legs; _name = name; }
  char* GetName() {return _name;}
  void Speak() {cout << "..." << endl; }
};
```




```
class Dog : public Animal{
public:
  Dog(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Guau" << endl; }
};
```

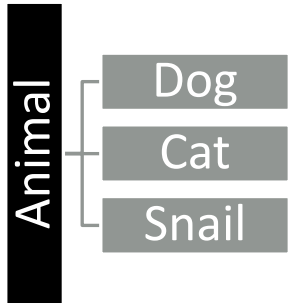

```
class Cat : public Animal{
public:
  Cat(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Miau" << endl; }
};
```

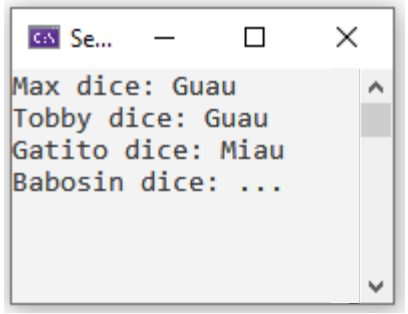

```
class Snail : public Animal{
public:
  Snail(int legs, char* name) :Animal(legs, name){}
};
```



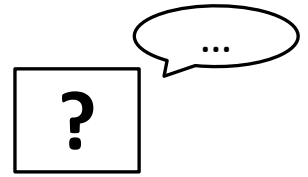
Inheritance without polymorfism


```
void main()
{
  Dog animal_1(4, (char*)"Max");
  Dog animal_2(3, (char*)"Tobby");
  Cat animal_3(4, (char*)"Gatito");
  Snail animal_4(0, (char*)"Babosin");

  //Animal animal_list[10] // OPTION 1
  Animal *animal_list = new Animal[10]; // OPTION 2
  animal_list[0] = animal_1;
  animal_list[1] = animal_2;
  animal_list[2] = animal_3;
  animal_list[3] = animal_4;
  for (int i = 0; i < 4; i++)
  {
     cout << animal_list[i].GetName() << " dice: " ;
     animal_list[i].Speak();
  }
}
```





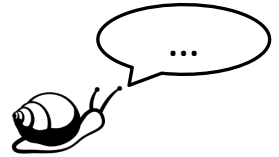

```
class Animal{
protected:
   char *_name;
   int _legs;
   int _type;
public:
   Animal(int legs = 0, char* name=(char*)""){ _legs=legs; _name=name; _type = 0;}
   char* GetName() {return _name;}
   void Speak() {
      switch (_type){
        case 1:cout << "Guau" << endl; break;
        case 2:cout << "Miau" << endl;break;
        default: cout << "..." << endl; break;
      }
}
};
```




```
class Dog : public Animal{
public:
   Dog(int legs, char* name) :Animal(legs, name) {_type = 1;}
   void Speak() { cout << "Guau" << endl; }
};
```


```
class Cat : public Animal{
public:
   Cat(int legs, char* name) :Animal(legs, name) {_type = 2;}
   void Speak() { cout << "Miau" << endl; }
};
```


```
class Snail : public Animal{
public:
   Snail(int legs, char* name) :Animal(legs, name) {_type = 3;}
};
```



Polymorphism and virtual functions

- The function ***speak()*** must act correctly with each object of the ***classes derived*** from the ***Animal*** class, without having to pass the object type.

It can be done:

1. with ***generic pointer*** a la clase base ***Animal***:

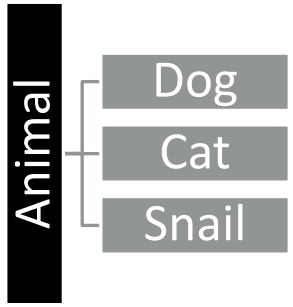
```
Animal* p_animal;
p_animal->Speak();
```

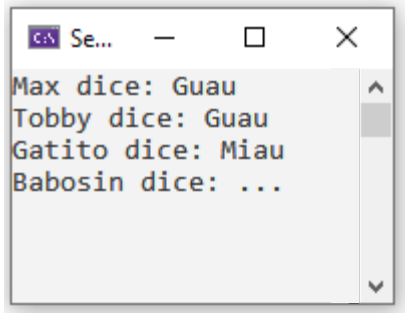
2. setting ***Speak()*** as a ***virtual function*** in the base class ***Animal***.
   - If this function is NOT declared as ***virtual***, the program will always use the function of the base class ***Animal ::Speak()***.

If a class does not have a function defined, it uses the one inherited from the ***base class***.

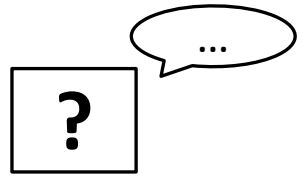
Polymorphism


```
void main()
{
  Dog animal_1(4, (char*)"Max");
  Dog animal_2(3, (char*)"Tobby");
  Cat animal_3(4, (char*)"Gatito");
  Snail animal_4(0, (char*)"Babosin");


  Animal* animal_list_2[10];
  animal_list_2[0] = &animal_1;
  animal_list_2[1] = &animal_2;
  animal_list_2[2] = &animal_3;
  animal_list_2[3] = &animal_4;
  for (int i = 0; i < 4; i++)
  {
     cout << animal_list_2[i]->GetName() << " dice: ";
     animal_list_2[i]->Speak();
  }
}
```





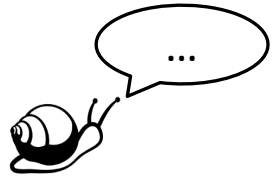

```
class Animal{
protected:
  char *_name;
  int _legs;
public:
  Animal(int legs = 0, char* name=(char*)""){ _legs = legs; _name = name; }
  char* GetName() {return _name;}
  virtual void Speak() {cout << "..." << endl; }
};
```




```
class Dog : public Animal{
public:
  Dog(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Guau" << endl; }
};
```

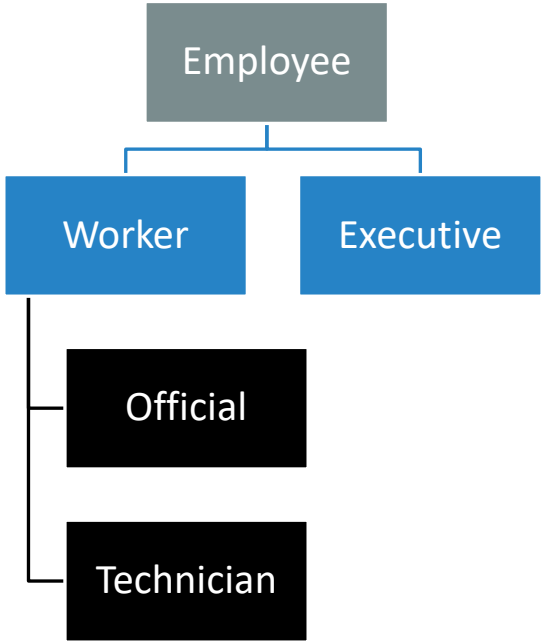

```
class Cat : public Animal{
public:
  Cat(int legs, char* name) :Animal(legs, name) {}
  void Speak() { cout << "Miau" << endl; }
};
```


```
class Snail : public Animal{
public:
  Snail(int legs, char* name) :Animal(legs, name){}
};
```



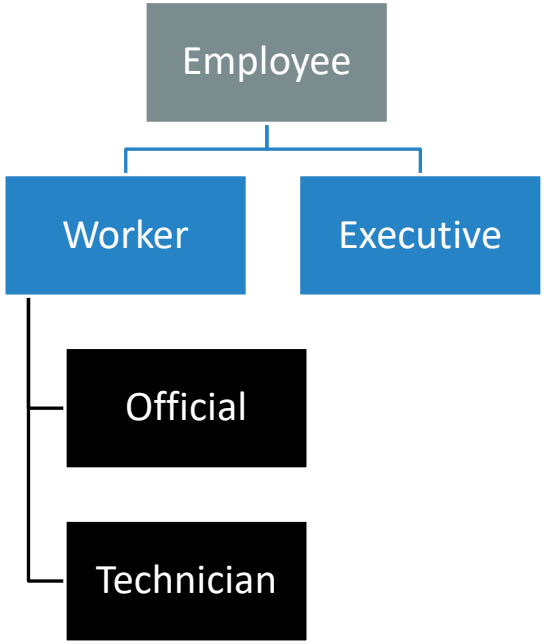
Example 1


```
//File Employee.h
#include <iostream>
using namespace std;

class Employee {
public:
  virtual void print_position(){
       cout << "Not defined\n"; }
};
class Executive : public Employee {
public:
  void print_position() {
      cout << "is an executive\n"; }
};
class Worker : public Employee {
public:
  void print_position() {
      cout << "is a worker\n"; }
};
class Official : public Employee {
public:
  void print_position() {
      cout << "is an official\n"; }
};
class Technician : public Employee {
public:
  void print_position() {
      cout << "is a technician\n"; }
};
```



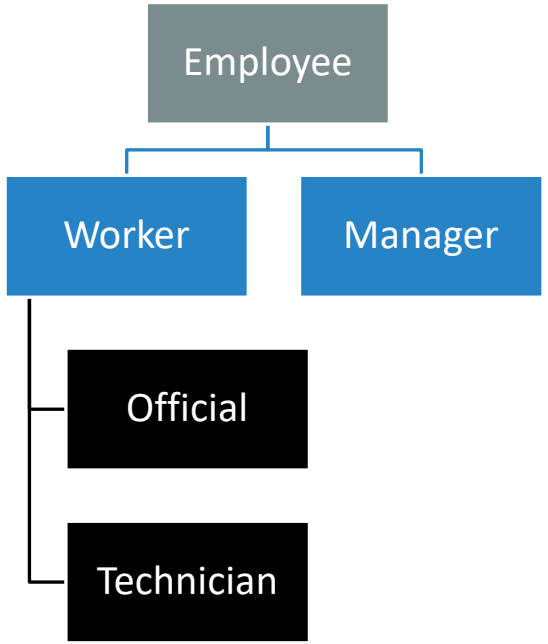
Example 1


```
// File Source.cpp
#include "Employee.h"
#include <string.h>
void main(void) {
    Employee Mary;
    Executive Steve;
    Worker Paul;
    Official Bob;
    Technician Ann;

    // The type of the object determines which function is called
    cout << "First with object names:" << endl;
    cout << "Mary "; Mary.print_position();
    cout << "Steve "; Steve.print_position();
    cout << "Paul "; Paul.print_position();
    cout << "Bob "; Bob.print_position();
    cout << "Ann "; Ann.print_position();

    Employee* pe;
    cout << "\n and now using pointers:" << endl;
    pe = &Mary;       cout << "Mary  ";     pe->print_position();
    pe = &Steve;      cout << "Steve ";     pe->print_position();
    pe = &Paul;       cout << "Paul ";      pe->print_position();
    pe = &Bob;        cout << "Bob ";       pe->print_position();
    pe = &Ann;        cout << "Ann ";       pe->print_position();
}
```



Example 2 – Implicit Polymorphism


```
void main(){
  Employee e((char*)"Steve", 1000);
  Manager m((char*)"Mary",1000,
(char*)"CEO");
  Worker w((char*)"Paul", 1000,
(char*)"Carpenter");
  Official o((char*)"Sarah", 1000,
(char*)"Architect");

  e.print_position();
  m.print_position();
  w.print_position();
  o.print_position();
}
```




```
class Employee {
protected:
    char* name;
    long salary;
public:
    Employee(char* n = (char*)"", long s = 0) {name = n;  salary = s; }
    virtual double CalculateFullSalary()  { return salary;  }
    void print_position() {
        cout << "Not defined\n Salary: " << this-> CalculateFullSalary()<<endl;
    }
};
```

```
class Manager : public Employee {
    char position[20];
public:
    Manager(char* n = (char*)"", long s=0, char* p =(char*)""): Employee(n, s) {strcpy_s(position, p);}
    double CalculateFullSalary()  {  return salary*1.2;  }
};
```

```
class Worker : public Employee {
protected:
    char title[20];
public:
    Worker(char* n = (char*)"", long s = 0, char* t = (char*)"") :Employee(n, s) {strcpy_s(title, t);}
    double CalculateFullSalary()  { return salary+100; }
};
```

```
class Official : public Worker {
public:
    Official(char* n = (char*)"", long s = 0, char* t = (char*)"") :Worker(n, s, t) {}
    double CalculateFullSalary() { return salary*1.05 + 20;  }
};
```

Pure virtual functions

Any ***virtual function*** must be defined at the base class, although it is never used in the base class

- because there are no objects of that class,
- because each derived class has its own definition of that function.

A ***virtual function*** of the base class that will never be used does not need to be defined: it is enough to declare it as a ***pure virtual function***. A pure virtual function is declared following next structure:

```
virtual float calculate_payment() const = 0;
```

In this case:

- It is not necessary to define the code of ***employee::calculate_payment()***.
- ***Objects of the base class employee*** can NOT be defined, as ***pure virtual functions*** cannot be called.
- ***Pointers to the employee class*** can be defined, as they can be used to handle objects of derived classes.
- It is necessary to redefine the function as const

Classes that declare ***pure virtual functions*** are called ***abstract classes***, as they do not have concrete objects of that class.

- If a derived class does not redefine an inherited ***pure virtual function***, the derived class inherits it as a ***pure virtual function*** and becomes an ***abstract class*** as well.

Ejemplo clase abstracta

```cpp
#include <iostream>
void main() {
  // IMPORTANTE: No se puede instanciar la clase Figura porque es
abstracta:
  // Figura f; // Esto daría un error de compilación.

  // Creamos objetos de las clases concretas
  Circulo c(5.0);
  Rectangulo r(4.0, 6.0);

  // Creamos un puntero a la clase base (polimorfismo)
  Figura* figuras[2];
  figuras[0] = &c;
  figuras[1] = &r;
  cout << "El area del circulo es: " << figuras[0]->calcularArea()
<< endl;
  cout << "El area del rectangulo es: " << figuras[1]-
>calcularArea() << endl;
}
```

```cpp
// Clase Base Abstracta
class Figura {
public:
    // **Función Virtual Pura**
    virtual double calcularArea() const = 0;

    // Un destructor virtual siempre es una buena práctica en clases base
    virtual ~Figura() {}
};
// Clase Derivada 1: Círculo
class Circulo : public Figura {
private:
    double radio;
public:
    Circulo(double r) : radio(r) {}
    double calcularArea() const override {
        // La palabra 'override' es opcional pero altamente recomendada
        // para asegurar que realmente estás sobrescribiendo una función
base.
        return 3.14159 * radio * radio;
    }
};
// Clase Derivada 2: Rectángulo
class Rectangulo : public Figura {
private:
    double ancho;
    double alto;
public:
    Rectangulo(double a, double h) : ancho(a), alto(h) {}

    // Implementación **obligatoria** de calcularArea()
    double calcularArea() const override {  return ancho * alto;   }
};
```

Polymorphism and virtual functions

Normally, in C++ the call to a given function is established at compile time (***static*** or ***early binding***).

With ***virtual functions*** this is not possible, because it is not known at that moment what type of object the pointer will point to at the time of the call. The function to be called is determined at run-time (***dynamic*** or ***late binding***).

With ***virtual functions***, it is possible for a user to add new classes to a class hierarchy, without modifying or recompiling the original code (thus without needing to know the source files).

Virtual functions are less efficient than normal functions:

- each class that uses ***virtual functions*** has a vector of pointers (one for each ***virtual function***) called***v-table.***
- each of these pointers points to the ***virtual function*** that is appropriate for that class, in principle to the definition itself if it exists.
- if a class does not have its own ***virtual function*** definition, the pointer points to the closest ***virtual function*** of its base class that has its own definition.
- Each object contains a hidden pointer to the ***v-table*** of its class. With the pointer to the object you access the ***v-table*** of the class and through it you access the appropriate definition of the ***virtual function***. This is the extra work.